Machine Unlearning on a Transformer: GA vs GradDiff vs KL

Oct 26, 2026 · @Samarth Prabhu K

TL;DR

We taught a small language model (Qwen2.5-0.5B) facts about 2 fake authors, then tried to make it forget them using three methods: **Gradient Ascent (GA)**, **Gradient Difference (GradDiff)** and **KL-regularized GA (KL)**.

All three share one idea: run training *backwards* on the facts to forget. They differ only in what stops the model from forgetting everything else. GA has no brake, GradDiff anchors on correct answers for other data, and KL anchors on the original model's whole output distribution.

What is machine unlearning?

Machine unlearning = removing specific knowledge from a trained model **without retraining it from scratch**.

Why people need it:

- **Privacy law**: GDPR's "right to be forgotten" means a user can demand their data be removed, and that includes from models.
- **Copyright**: a model trained on a book may need to stop reproducing it.
- **Safety**: remove dangerous knowledge, such as bioweapon recipes.

Why not just retrain? Retraining without the data is the perfect answer, but for a big LLM it costs months and millions of dollars. Unlearning tries to get close to that "retrained" model in a few minutes of fine-tuning.

Analogy: Retraining is rebuilding the whole library without one book. Unlearning is finding that book's pages scattered across millions of shelves and erasing them, without smudging the books next to them. The smudging is the hard part.

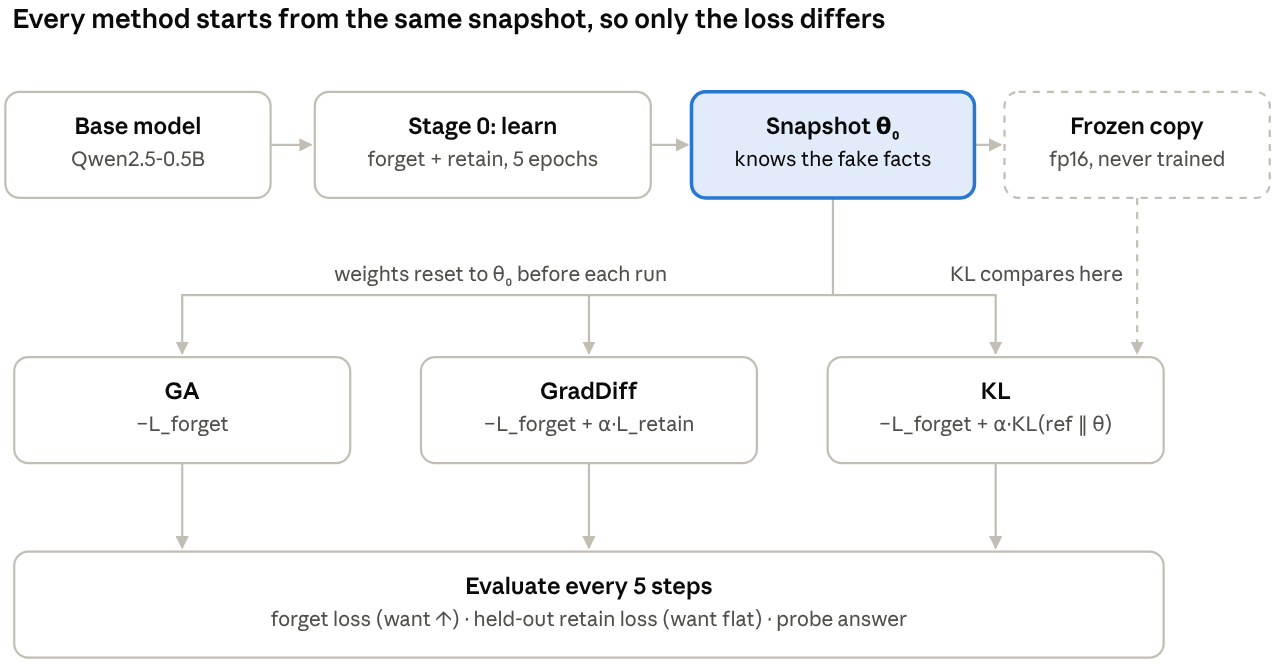
Our setup

The run had two stages: first make the model **know** the facts, then make it **forget** them.

Piece	What we used	Why
Model	Qwen2.5-0.5B (decoder-only transformer, ~0.5B params)	Small enough to fully fine-tune on a free Colab T4 GPU
Dataset	TOFU: Q&A about 200 fictitious authors	The authors are fake, so we know for sure the model only learned them from us
Forget set	`forget01` : 40 Q&A pairs about 2 authors	The knowledge we want gone
Retain set	200 Q&A pairs about other authors	Used *during* unlearning to protect other knowledge
Held-out retain	100 more Q&A pairs	Never touched during unlearning, so it gives an honest check that nothing else broke

Why fake authors? If we used real facts (say, "Who wrote Hamlet?"), the model already knew them from pre-training. We couldn't tell whether it forgot *our* training or had never relied on it. Fake data gives a clean before/after.

Loss is measured only on the **answer** tokens, not the question. We want to know how well it produces the answer, not how well it predicts the question.



experiment pipeline · learn, snapshot, 3 unlearning runs

Resetting to the same snapshot before each run makes the comparison fair: the only thing that differs between GA, GradDiff and KL is the loss.

The core concept: running training backwards

A language model predicts the next token. Its loss on an answer is the **negative log-likelihood (NLL)**, i.e. how surprised the model is by each correct token:

```
\mathcal{L}(\theta; x, y) = -\frac{1}{|y|}\sum_{t=1}^{|y|} \log p_\theta(y_t \mid x, y_{<t})
```

Here x = the question, y = the answer, θ = the model's weights.

- Low loss: the model is confident in the correct answer, so it **knows** the fact.
- High loss: the correct answer looks unlikely to the model, so it has **forgotten** it.

Normal training (gradient descent) moves weights downhill on the loss:

```
\theta \leftarrow \theta - \eta \, \nabla_\theta \mathcal{L}
```

Unlearning (gradient ascent) flips the sign and moves uphill on the forget data:

```
\theta \leftarrow \theta + \eta \, \nabla_\theta \mathcal{L}_{\text{forget}}
```

In code, this is just `loss = -f_loss`. The optimizer still minimizes, but minimizing the negative is the same as maximizing the original.

Analogy: Training is carving a groove so a marble rolls to the right answer. Gradient ascent is pushing the marble back up out of the groove. The catch: the same weights hold grooves for *everything else* too, so pushing hard enough flattens the whole landscape.

Method 1: Gradient Ascent (GA)

GA maximizes loss on the forget set and does nothing else.

```
\min_\theta \; -\mathcal{L}_{\text{forget}}(\theta)
```

Intuition: Every step makes the forget answers less likely. It works fast: forget loss shoots up within a few steps.

Failure mode: catastrophic collapse. The objective has no ceiling. Loss can grow forever, because the model can push the probability of the right token toward 0 indefinitely. The cheapest way to do that is to wreck the model in general, so after enough steps it outputs gibberish or repeats one token for *every* question, forget and retain alike.

That's why we clip gradients (`clip_grad_norm_ = 1.0`), use a tiny learning rate (1e-5) and stop at 30 steps. These are hand brakes, not a fix.

Method 2: Gradient Difference (GradDiff)

GradDiff does ascent on the forget set **and** normal descent on the retain set, in the same step.

```
\min_\theta \; -\mathcal{L}_{\text{forget}}(\theta) +
\alpha\,\mathcal{L}_{\text{retain}}(\theta)
```

α (we used 1.0) sets how hard the retain side pulls.

Intuition: a tug-of-war on the same weights. The forget term drags the weights away from the forget facts. The retain term drags them back toward "still answer other authors correctly". Weights that only matter for the forget facts lose the tug-of-war and get erased. Weights that are shared get held in place.

Weakness: The retain term only protects the *one correct token*. The model can still drift on everything else it outputs (its confidence, its second-best guesses) as long as the right answer stays on top.

Method 3: KL-regularized GA (KL)

KL does ascent on the forget set, plus a penalty for changing the model's **whole output distribution** on retain data compared to a frozen copy of the original model.

```
\min_\theta \; -\mathcal{L}_{\text{forget}}(\theta) +
\alpha\,\mathrm{KL}\big(p_{\text{ref}}(\cdot \mid x_r)\,\|\,p_\theta(\cdot
\mid x_r)\big)
```

```
\mathrm{KL}(P\,\|\,Q) = \sum_{v \in \text{vocab}} P(v)\,\log\frac{P(v)}{Q(v)}
```

KL divergence measures how different two probability distributions are. It is 0 only when they are identical.

Why this is stronger than GradDiff: At each position the model outputs a probability over all ~150k vocabulary tokens, not one answer.

	GradDiff retain term	KL retain term
Compares against	The one correct token	The original model's full distribution
Protects	"Right answer stays on top"	The model's whole behaviour, including confidence and alternatives
Needs	Only data	Data + a frozen copy of the model (more memory)

Analogy: GradDiff says "keep getting the exam answers right". KL says "keep *thinking* exactly like you used to, everywhere except the forget topic".

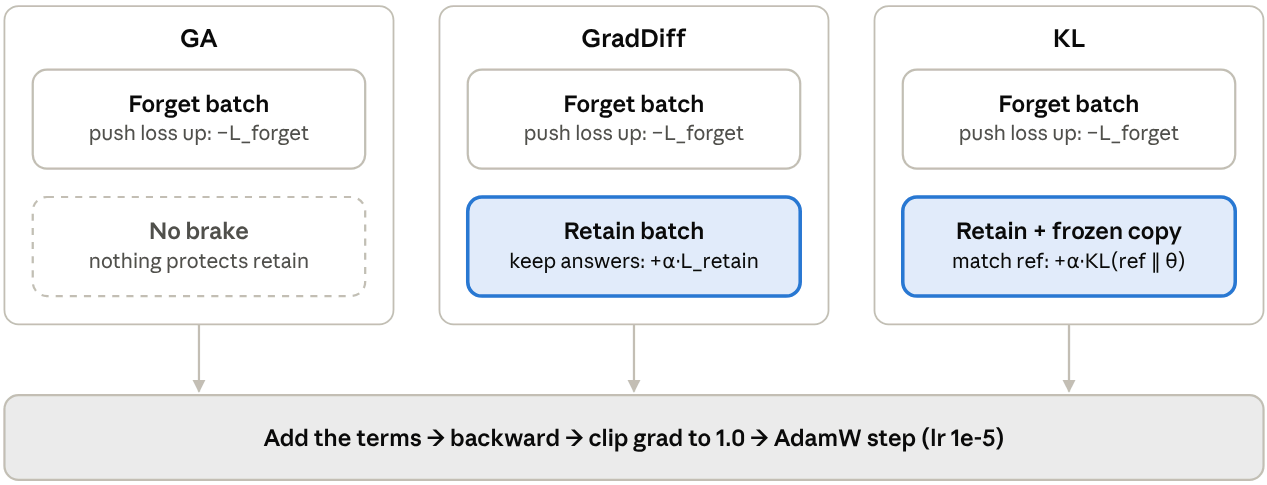
We keep the frozen reference in fp16 (`ref_model.half()`) to save GPU memory. It is only read, never trained.

The three methods side by side

All three share the same forget term; only the brake changes.

	GA	GradDiff	KL
Loss	−L_forget	−L_forget + α·L_retain	−L_forget + α·KL(ref ‖ θ) on retain
Brake against collapse	None	Correct answers on retain data	Original model's full output on retain data
Data needed	Forget set	Forget + retain	Forget + retain
Extra memory	None	None	Frozen copy of the model
Expected forget loss	Rises fastest	Rises	Rises
Expected retain loss	Rises too (collapse)	Roughly flat	Roughly flat, most stable
Code	`loss = -f_loss`	`-f_loss + alpha * retain_loss`	`-f_loss + alpha * kl`

Same forget term in all three; only the brake under it changes



one unlearning step · 3 methods, batch size 4

The highlighted boxes are the brakes. GA has none, which is why it collapses first.

How to read our results

Good unlearning means **forget loss goes up while held-out retain loss stays where it was**. Either number alone means nothing: GA with 1000 steps "forgets" perfectly because it forgot how to speak.

The notebook prints three things:

1. **Results table**: forget loss, held-out retain loss and the model's answer to one forget question, for the original model and each method.
2. **Two plots over steps**: forget loss (want ↑) and held-out retain loss (want flat).
3. **Probe answer**: what the model now says to a forget question.

How to judge the probe answer:

Output	What it means
Still the true answer	Didn't forget, so we need more steps or a higher learning rate
A fluent but wrong answer	Best case: it forgot and still speaks normally
Gibberish or a repeated token	Collapsed: "forgot" by breaking

Paste your numbers from the results table here:

Method	Forget loss	Held-out retain loss	Probe answer
Original			
GA			
GradDiff			
KL			

Limitations: what this does not prove

High forget loss shows the model stopped *saying* the answer, not that the knowledge is gone from its weights.

- **Paraphrase test**: ask the same fact in different words. Surface-level unlearning often fails here.
- **Relearning attack**: fine-tune the unlearned model on a few forget examples. If it recovers in 5 steps instead of 50, the knowledge was hidden, not removed.
- **Probing**: train a small classifier on hidden activations to detect the fact. If it still works, the information is still inside.
- **Gold standard comparison**: a model retrained without the forget set. TOFU's real metric (forget quality) compares against exactly this.
- **Scale**: one run, one seed, 40 forget examples and 30 steps. Treat the result as a demo, not evidence.

Next steps

The natural next method is **NPO (Negative Preference Optimization)**. It fixes GA's unbounded loss by squashing it through a log-sigmoid, so the push fades as the model forgets:

```
\mathcal{L}_{\text{NPO}} = \frac{2}{\beta}\,\mathbb{E}\left[\log\left(1 +
\left(\frac{p_\theta(y \mid x)}{p_{\text{ref}}(y \mid
x)}\right)^{\beta}\right)\right]
```

- [ ] Run GA for 100 steps and watch retain loss explode
- [ ] Sweep $\alpha \in \{0.1, 1, 5\}$ for GradDiff and KL
- [ ] Ask paraphrased forget questions
- [ ] Add NPO to the notebook
- [ ] Port the same three methods to Mamba (our capstone): the loss functions are architecture-agnostic, only the model name changes

Questions to discuss with the group:

1. Why does NPO's loss stop growing, when GA's doesn't? Look at what happens to the formula as $p_\theta \to 0$.
2. In Mamba, knowledge lives in a recurrent state, not attention. Would GA erase it the same way, or does the state carry it somewhere gradients reach differently?